Compact, Interleaved, Portable: An Open Reference Implementation of the Batched Small-Matrix Linear-Algebra API

A cross-vendor performance study against Intel MKL Compact and Arm Performance Libraries

Ivan Pribec*

August 6, 2026 ▶ [preprint draft — not for circulation]

Abstract

Many scientific kernels do not factor one large matrix but millions of tiny ones, all the same shape: the local systems of meshless approximation, the mapping problems of multi-physics coupling, per-cell chemistry, and per-column implicit solves. The leading vendor answer is the *compact*, or *interleaved-batch*, storage format, in which element (i, j) of V matrices is laid out contiguously so that a single SIMD instruction advances V independent factorizations at once. Intel MKL (Compact) and Arm Performance Libraries (interleave-batch) both ship such kernels — but each is closed, locked to one instruction-set family, and, in MKL's case, incomplete: it can factor and triangular-solve a batch but offers no supported way to *apply* the orthogonal factor Q. We present `cqr`, an open reference implementation of the compact API written as a *single* vector-typed source that the compiler lowers to SSE, AVX, AVX-512 and, in progress, NEON/SVE. It is a drop-in alternative to `mkl_?geqrf_compact`, it supplies the missing `?ormqr_compact` apply-Q step, and it is intended to interoperate with a vendor's own compact routines. We describe the format and the two vendor APIs, the portable-SIMD design and its one non-trivial kernel (a branch-free `larfg`), and benchmark the batched QR factorization against MKL Compact on x86 and against ArmPL interleave-batch on Arm. ▶ [one-sentence headline result once measured runs are in.]

1 Introduction

Dense linear algebra is usually told as a story about *one* large matrix: block the operation, reuse the cache, saturate the vector units. A large and growing class of applications tells the opposite story. They must factor or solve a *batch* of many small matrices — orders of a few up to a few hundred — that share a common shape, and no single one of them is large enough to keep a modern core busy. Calling a tuned LAPACK routine once per matrix wastes the machine: the matrices are too small to amortize call overhead or to vectorize internally, and the loop leaves the vector lanes mostly idle.

1.1 Motivating applications

Two applications motivate this work directly; several others share its shape.

* ▶ [confirm author list, affiliation, e-mail, ORCID; add co-authors and acknowledgements.]

Meshless approximation: MLS and RBF-FD. Moving least squares (MLS) and radial-basis-function finite differences (RBF-FD) build differential operators on *scattered* point clouds without a mesh [6, 14]. Around each node one gathers a local stencil of its k nearest neighbours and solves a small dense problem for that node's differentiation weights: a (weighted) least-squares fit of a polynomial or RBF basis in the MLS case, or a saddle-point RBF-plus-polynomial interpolation system in the RBF-FD case. A discretization with N nodes produces N such systems — routinely 10^6–10^8 of them — *all of the same order*, set by the stencil size. Typical stencils of 30–170 points in 2-D and 3-D are exactly the non-power sizes $(30, 45, 60, 105, 168)$ exercised by our benchmark (Section 5). This is the canonical batched-small-matrix workload, and it is the application we care about most.

Data mapping for multi-physics coupling. Partitioned multi-physics couples separately discretized solvers across a shared interface, and their surface meshes rarely match. Coupling libraries such as preCICE [5] therefore *map* field data between non-matching point sets, and radial-basis-function interpolation is a standard choice. Local or partition-of-unity RBF mappings assemble and solve one small dense system per output point or per partition — again many systems of a common size, re-solved every time the interface geometry changes. The batched compact format is a natural fit for the assembly-and-solve inner loop.

Others of the same shape. The pattern recurs. Implicit *chemical kinetics* in reacting-flow solvers integrates a stiff ODE per cell, whose Newton step is a small dense solve of the species Jacobian — one per cell, identically sized. ▶ [add a kinetics reference (e.g. a GPU/batched chemistry paper).] *PDE line-solvers* — ADI sweeps, implicit vertical transport in atmosphere/ocean columns — produce one small (often banded, sometimes dense) system per grid line. And sequential *convex optimization* (interior-point, SQP, model-predictive control) repeatedly solves small dense KKT systems; the GPL-licensed `batmat` project [11] targets exactly this domain with interleaved batched kernels. ▶ [add an optimization/MPC reference.] What unites them is not the mathematics but the memory pattern: a large batch of identically shaped small matrices, ripe for cross-matrix vectorization.

1.2 The compact (interleaved) format

The vendor response is a storage format built for this pattern. Rather than store the V matrices of a group back to back, the *compact* (Intel) or *interleaved-batch* (Arm) format *interleaves* them, so that the V copies of element (i, j) sit in V contiguous words. A scalar factorization kernel then lifts almost verbatim to SIMD: replace `double` with a V-wide vector type and every arithmetic operation advances V independent matrices in lockstep (Section 2). Intel MKL exposes this through its Compact BLAS/LAPACK functions [8]; Arm Performance Libraries through its interleave-batch functions [3]. Both grew out of the wider batched-BLAS effort [1, 4].

1.3 The gap

Useful as they are, the vendor kernels share three limitations. They are *closed*, so they cannot be studied, ported, or tuned for a new size regime. They are *locked to one ISA family* — MKL Compact to x86, ArmPL to AArch64 — even though the interleaved idea is architecture-neutral. And MKL's compact LAPACK is *incomplete* for the QR pipeline: it provides QR, Cholesky, and unpivoted-LU factorizations and a triangular solve, yet neither `?ormqr` nor `?orgqr` — no supported way to apply or form the orthogonal factor Q, the step that turns a QR factorization into a least-squares solve.

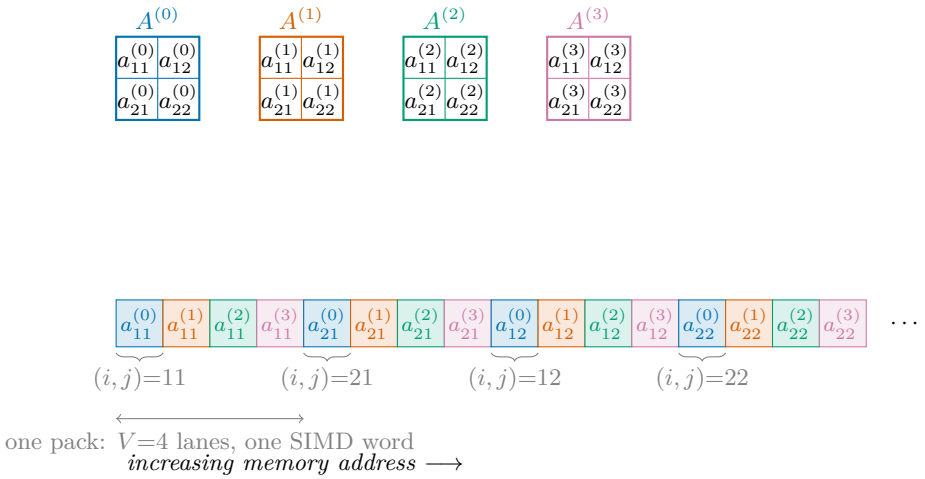

Figure 1: The compact / interleaved-batch layout for a pack of V=4 matrices of order 2 (column-major). The V copies of each element (i, j) occupy V contiguous words, so a single V-wide SIMD load gathers the same element of all V matrices, and a scalar kernel becomes a batched one by widening its scalars to vectors.

1.4 Contributions

This paper presents `cqr`, an open, portable reference implementation of the compact API, and studies its performance against both vendors. Specifically:

- We position the compact/interleaved format and lay the two vendor APIs side by side, making the portability and completeness gaps explicit (Sections 2–3).
- We describe a portable-SIMD implementation from a *single* source — GNU vector types lowered by the compiler to SSE/AVX/AVX-512 (and, in progress, NEON/SVE) — and its one genuinely hard kernel, a branch-free vectorized `larfg` (Section 4).
- We supply the apply-Q step (`cqr_mkl_?ormqr_compact`) that MKL omits, completing the compact QR solve pipeline.
- We benchmark the batched QR factorization against MKL Compact on x86 and ArmPL interleave-batch on Arm, over the small-size range that the applications above actually use (Sections 5–6).

`cqr` is real-precision (single and double) and unpivoted by design; its scope limits mirror the vendors' own (Section 7).

2 The compact (interleaved) format

Consider a batch of n_m matrices, all $m \times n$. Group them into $\lceil n_m/V \rceil$ *packs* of V matrices, where V is the SIMD width (for double precision, $V = 2/4/8$ for SSE/AVX/AVX-512). Within a pack, the compact format stores the V values of a given element contiguously; consecutive elements (in the layout's natural order — column-major here) follow one after another. Figure 1 shows a pack of $V = 4$ matrices of order 2.

This differs from the other common batched convention, the *pointer array* of BBLAS [1], in which each matrix keeps its ordinary dense layout and the batch is an array of pointers. Pointer arrays impose no repacking and suit routines (like batched GEMM) whose per-matrix work is already large enough to vectorize internally; the interleaved layout instead extracts parallelism *across* matrices, which is precisely what wins when each matrix is far too small to

Table 1: Coverage of the compact / interleaved batched APIs. Routine stems are shown without the precision letter and the vendor's suffix (MKL `mkl_?X_compact`; ArmPL `armpl_?X_interleave_batch`). ✓ = provided, — = absent. Variant tags: NP no pivoting, TP threshold pivoting, RR rank-revealing. Packing is MKL `?gepack`/`?geunpack` (with `get_format` / `get_size`) versus ArmPL `?ge_interleave`/`?ge_deinterleave` (strides passed explicitly).

Operation	MKL Compact	ArmPL interleave-batch	`cqr`
BLAS			
`dot`, `scal`, `ger`, `gemv`	—	✓	—
`gemm`	✓	✓	—
`trmm`, `trsv`	—	✓	—
`trsm`	✓	✓	—
LAPACK			
Cholesky `potrf`	✓	✓	design
LU `getrf`	✓ NP	✓ TP	—
LU inverse `getri`	✓ NP	—	—
QR `geqrf`	✓	✓ RR	✓
form Q `orgqr`	—	✓	—
apply Q `ormqr`	—	✓	✓

fill a vector register on its own. The cost is a pack/unpack step at the batch boundary (MKL's `mkl_?gepack_compact/geunpack_compact`); in the target applications the same factored batch is reused enough — across right-hand sides, time steps, or Newton iterations — to amortize it. Parallelism can also come from an execution policy — one matrix per thread or team — rather than the data layout, and the cross-matrix SIMD of the interleaved approach can be had through a vector *element type* instead of a dedicated storage format; Kokkos Kernels offers both (Section 8).

3 Two vendor APIs, side by side

Intel MKL Compact. MKL hides the interleave width behind an opaque `MKL_COMPACT_PACK` token from `mkl_get_format_compact()`, with `?gepack_compact` / `?geunpack_compact` moving data in and out [8]. Its kernels (Table 1) are compact GEMM and TRSM, Cholesky, an *unpivoted* LU and its inverse (`?getrfnp`, `?getrinp`), and QR (`?geqrf`). The LU is unpivoted by name and by design: partial pivoting's per-column search and row interchange do not vectorize cleanly across a pack, so MKL forgoes them. The conspicuous hole is in QR — MKL ships neither `?ormqr` nor `?orgqr`, so a caller can obtain the reflectors (H, τ) and triangular-solve, but has no supported way to *apply* Q^{T} to a right-hand side, the step that turns the factorization into a least-squares solve. MKL's compact routines also skip argument checking "for performance reasons" and leave `info` reserved [8], with documented limits near underflow/overflow and no pivoting [9].

Arm Performance Libraries interleave-batch. ArmPL exposes the same idea with a more explicit hand: the caller passes inner, outer, and batch strides describing the interleave directly, and `?ge_interleave` / `?ge_deinterleave` pack and unpack [3]. Its coverage is the broadest of the three (Table 1): the BLAS-1/2/3 building blocks (`dot`, `scal`, `ger`, `gemv`, `gemm`, `trmm`, `trsv`, `trsm`) and a *complete* QR pipeline — rank-revealing factorization (`?geqrfrr`), `?orgqr` to form Q, and `?ormqr` to apply it — besides Cholesky and LU. Alone among the three it pivots at all in the compact setting: its LU uses *threshold* pivoting (`?getrftp`). In the usual sense of the term, the in-place (diagonal) pivot is accepted while it exceeds a fixed fraction of the largest candidate in its column, and a row interchange is taken only when it does not — so most lanes stay on the cheap no-swap path and only the ill-conditioned ones diverge, trading a little stability for a

Table 2: Design choices behind the four batched libraries — the "how", complementing the "what" of Table 1. (Kokkos Kernels is not a compact API, so it is absent from Table 1 but belongs here.)

Design choice	MKL Compact	ArmPL interleave-batch	Kokkos Kernels	`cqr`
API / dispatch	C ABI, runtime	C ABI, runtime	C++ templates, compile-time	C ABI + templated kernels
Storage	interleaved	interleaved	per-matrix or SIMD-packed Views	interleaved
Cross-matrix work	SIMD lanes	SIMD lanes	SIMD value type and/or threads (Serial/Team/TeamVector)	SIMD lanes
Callable from	C, C++, Fortran, Python	C, C++, Fortran, Python	C++	C, C++, Fortran, Python
Precisions	s, d, c, z	s, d, c, z	s, d, c, z	s, d
Target hardware	x86	AArch64	CPU + GPU	any CPU SIMD
Source	closed	closed	open	open

batch pattern that stays mostly uniform. ▶ [confirm ArmPL's exact `getrftp` pivoting semantics against their reference before publication.]

`cqr`. `cqr` adopts MKL's convention — the opaque `MKL_COMPACT_PACK` token and the identical argument list — so a call site can swap `mkl_?geqrf_compact` for `cqr_mkl_?geqrf_compact` unchanged, feed the result to MKL's `?trsm_compact`, and use `cqr_mkl_?ormqr_compact` for the apply-Q step MKL lacks; a second, MKL-free C API takes the width V explicitly for use without MKL. Its scope is deliberately narrow (Table 1): the QR factorization, the apply-Q that completes the solve, and a designed-but-unbuilt Cholesky. It provides no batched GEMM — the one kernel that already vectorizes well per matrix and that the vendors tune hard, so there is little to gain from re-implementing it — and no LU, since the pivoting question (forgo it like MKL, or threshold it like ArmPL) is a study in itself. What `cqr` adds over both vendors is not breadth but openness and portability: one vector-typed source, no ISA lock (below).

Dispatch: a C ABI at runtime, or C++ templates at compile time. Intel and Arm ship *raw C functions*. That buys ABI stability and callability from C, C++, Fortran, and Python, at the price of resolving the storage format and vector width *at runtime*, inside a precompiled binary. Kokkos makes the opposite choice: the properties and the algorithm are C++ template parameters resolved at *compile time*, maximizing inlining and fusion but restricting the caller to C++ and requiring a recompile per configuration. Neither is strictly better — the runtime indirection is a fixed cost that matters most when the work per call is small, and whether a compile-time-specialized library wins in that regime is exactly the open question. `cqr` takes a hybrid position (Table 2): it keeps the vendor's C ABI — so it stays a drop-in and remains callable from Fortran and Python — but resolves the pack format to a compile-time interleave width *once per call* and dispatches to a kernel fully specialized on (T, V), so the inner sweep carries no runtime branch. The runtime choice is paid once per batch, not once per element.

Parallelism granularity. Two orthogonal levers set how a batch maps to the hardware, and it helps to keep them apart. *Across* matrices, a SIMD lane can carry a whole distinct matrix so V advance in lockstep — what the compact storage format delivers directly, and what Kokkos delivers through a SIMD value type (Section 8); either way the cross-matrix SIMD is there. *Within* a matrix, the work can be done by one worker or shared among several — the axis Kokkos exposes as `Serial` (one thread per matrix), `Team` (a team of threads per matrix), and

`TeamVector` (team threads plus their vector lanes per matrix). The compact libraries fix the first lever (one matrix per lane) and leave the batch loop to the caller's threads; Kokkos exposes both, and — being built on Kokkos — maps the second onto GPU threads, where CPU-style SIMD lanes do not exist. For the many-tiny-matrices regime targeted here, one matrix per lane and one matrix per thread are the natural CPU mappings; the `Team` levels earn their keep as matrices grow or thin out.

4 `cqr`: an open, portable reference implementation

One source, every width. `cqr`'s central decision is to write each kernel once, over a scalar type `T` and an interleave width `V`, using GNU vector types (`__attribute__((vector_size))`). The compiler lowers that single source to SSE, AVX, or AVX-512 on x86 — and to NEON or SVE on AArch64 — with no per-ISA intrinsics. A thin dispatcher maps the runtime `MKL_COMPACT_PACK` (or an explicit width) to the compile-time `V` that selects the specialization. This is what lets one code base stand in for two vendors' closed kernels.

Vectorized unblocked Householder QR. The factorization is the unblocked LAPACK algorithm `geqr2` (`larfg` to build each reflector, `larf` to apply it to the trailing columns [2, 7]), run V matrices at a time. Because the compact layout makes element (i, j) of all V matrices contiguous (Figure 1), the scalar algorithm lifts to SIMD by widening `double` to a V-wide vector: every $+, -, \times, \div$ becomes a lane-wise operation over V independent matrices. The *blocked* algorithm (`geqrf`, with `larft`/`larfb`) is deliberately *not* used: at these orders the reflector panels are short, and the interleaved batch already saturates the vector units without the extra bookkeeping. Column-major is the tuned path — a matrix column is contiguous in the compact buffer, so the norm reduction, the reflector scaling, and the trailing-column update all walk contiguous pointers, register-blocked a few trailing columns at a time.

The one hard part: a branch-free `larfg`. Scalar `larfg` contains a data-dependent branch (if the below-diagonal norm is zero, $\tau = 0$) and scalar special functions. Across a pack that branch would diverge per lane, so it is rewritten branch-free with a mask and a select, keyed on the LAPACK-correct quantity (the *below-diagonal* norm, so an already-triangular column yields $\tau = 0$ with the diagonal untouched, exactly as `larfg` does):

```
tail  = sum_{i>k} A(i,k)^2            // vector reduction below the diagonal
x0    = A(k,k)
norm  = sqrt(x0*x0 + tail)            // == hypot(x0, ||tail||)
beta  = (x0 >= 0) ? -norm : norm      // -copysign(norm, x0), branch-free
has   = tail > 0                      // lane mask: is there anything to zero?
tau   = has ? (beta - x0)/beta : 0
inv   = has ? 1/(x0 - beta)     : 0   // masked select kills 0/0 and 1/0
A(k,k)     = has ? beta : x0          // R diagonal (unchanged if no tail)
A(i>k,k)  *= inv                      // reflector body (0*inv = 0 when !has)
```

Only `sqrt` needs a helper — a short lane loop that both GCC and Clang lower to a single `vsqrt` — while the comparison yields a native mask and the select is a bit-blend. Every step is $O(1)$ per column; the $O(n^2)$ reductions and $O(n^3)$ trailing update are pure vector arithmetic. The same `has` mask that makes an already-triangular column a no-op also neutralizes the identity matrices that padding puts in the unused lanes of a partial final pack, so the kernel runs unmasked at full width without ever producing a `NaN`.

The remainder "staircase". Because work is organized around the width V and a small register-blocking factor, orders n that are not multiples of those quantities leave a partial tile

Table 3: `cqr` feature coverage. The tuned contiguous kernel serves the column-major, left-apply solver path; the other layout/side combinations run through a correctness-first stride-generalized kernel over the same math.

Precisions	single (`s`), double (`d`); real only
Layouts	column-major (tuned), row-major (stride-generalized)
`geqrf`	vectorized unblocked `geqr2`, branch-free `larfg`
`ormqr`	side $\in \{L, R\}$, trans $\in \{N, T\}$ (C folds to T)
`potrf`	designed (batched Cholesky); not yet implemented
Checking	none on the MKL-style API; LAPACK-style `info`$= -j$ on the portable C API
Not provided	LU, GEMM, and BLAS-1/2 — deferred to the vendor libraries
Not supported	complex (`c`,`z`); column pivoting; underflow/overflow rescaling

whose cleanup is proportionally more expensive at small n. The sustained rate therefore climbs in a gentle staircase rather than a smooth curve (Figure 5); the effect is most visible at the smallest orders and is exactly why the benchmark includes non-power sizes drawn from real stencils.

Completing the pipeline and its scope. With `cqr_mkl_?ormqr_compact` supplying the apply-Q step, the three routines factor and solve a batched least-squares/linear system end to end — `geqrf_compact` → `ormqr_compact`($'L',{}'T'$) → `trsm_compact` — the last borrowed from the vendor. Table 3 summarizes coverage. By design `cqr` is real-precision only, performs no argument checking on the MKL-style surface (like MKL), does not pivot (the row interchanges of pivoting diverge across a pack), and does not implement `larfg`'s underflow/overflow rescaling — the same limitation MKL documents for its compact QR [9].

5 Experimental methodology

▶ [This section states the intended protocol; fill the two machine specifications and confirm the details match the runs that produce the data.]

Machines. **x86 (provisional):** an AVX-512 Xeon (VNNI-capable) at 2.8 GHz, 4 vCPUs on a shared virtual machine; GCC 13.3 with `-O3 -march=native`; Intel MKL (Ubuntu `libmkl`). ▶ [shared, non-pinned VM — provisional only; replace with the empty pinned node's specs and re-run.] **Arm:** ▶ [AArch64 CPU, SVE vector length, cores, ArmPL version, compiler and flags.]

Workload. Each data point factors a pool of 512 random, well-conditioned square matrices of a fixed order n, in double precision. The size list spans the target range and deliberately mixes powers of two with the non-power stencil sizes $30, 45, 60, 105, 168$ (Section 1.1) so the remainder staircase is visible: $\{8, 16, 24, 30, 32, 45, 48, 60, 64, 96, 105, 128, 168, 170, 256, 384, 500\}$.

What is timed. The pool is packed into compact form *once*, up front; only the factorization is timed, and the destroyed input is restored (untimed) before each pass. Timing uses Google Benchmark: a warm-up, then 10 repetitions per point, reported as the median. The two compact paths (`cqr`, vendor) are driven from an outer parallel loop over packs — the intended usage — with the vendor library's own threading pinned to one thread; the per-matrix LAPACK baseline parallelizes over matrices the same way. This isolates the factorization *kernel*, not pack/unpack or data movement.

Correctness gate. Every timed configuration is checked (untimed) against per-matrix `LAPACKE_?geqrf`: the unpacked (H, τ) must match elementwise to a tight tolerance, so each benchmark row doubles as an integration test. Reported `relerr` is this quantity.

Roofline. The GFLOP/s figures overlay the node's attainable double-precision compute peak, measured with LIKWID [13] on the same node and core count as the runs (the `likwid-bench` peak-flops kernels; the kernel's achieved rate can also be read back with `likwid-perfctr -g FLOPS_DP`). Reading the achieved rate against that ceiling is a roofline view [15] of the factorization. For the provisional x86 machine the ceiling is 316.9 GFLOP/s (`likwid-bench -t peakflops_avx512_fma`, 4 threads, ∼88% of the 2.8 GHz AVX-512 FMA theoretical peak). ▶ [measure the Arm peak, and re-measure x86 on the pinned node at the run's core count.]

Reproducibility. The x86 numbers come from the report's Google Benchmark collector (`report/bench`, a separate CMake project); the ArmPL numbers require the AArch64 harness described in Section 9. Scripts, data, and this document build with a single `make` (see the report `README`).

6 Results

▶ [The x86 figures show **real measurements**, but from a shared, non-frequency-pinned virtual machine — treat them as *provisional* and re-run on the target node for the paper. The Arm figures remain **synthetic placeholders** pending hardware (Section 9). See `report/data/README.md`.]

Throughput on x86 (vs MKL Compact). Figure 2 plots throughput (matrices/s) against order for `cqr`, `mkl_?geqrf_compact`, and the per-matrix LAPACK loop; Figure 3 recasts it as speedups over the per-matrix baseline. Two effects stand out. First, the compact format's regime is the small sizes: both compact paths beat the per-matrix loop by a wide margin at the smallest orders (several-fold at $n \lesssim 32$), the advantage shrinking with n until, around $n \approx 128$, per-matrix LAPACK draws level and then overtakes — its internally blocked, cache-blocked `geqrf` scales into a regime the unblocked compact kernel is not meant for. This is the expected crossover, and it places the compact format exactly where the motivating applications live (stencils of a few tens up to ∼150 points). Second, `cqr` matches or beats MKL's own compact `geqrf` across essentially the whole range, so the open kernel costs nothing against the closed one on this machine. ▶ [quote the small-n speedup, the crossover order, and the cqr/MKL geometric mean from the pinned-node run.]

Throughput on Arm (vs ArmPL interleave-batch). Figure 4 repeats the comparison on AArch64 against ArmPL's interleave-batch `geqrf`. The question here is the *portability tax*: how close does a single vector-typed source come to a vendor kernel hand-tuned for SVE? ▶ [report the observed gap; note where `cqr` leads (smallest orders) and where ArmPL's tuning pulls ahead.]

Absolute rate against the roofline. Throughput ratios say who wins; they do not say how much performance is left on the table. Figures 5 and 6 therefore plot the achieved rate (GFLOP/s) of all three drivers — `cqr`, the vendor compact kernel, and the per-matrix LAPACK loop — against the machine's double-precision compute ceiling measured with LIKWID [13]. Three things to read off them. First, the compact paths lead in absolute GFLOP/s at the small orders but plateau early: the unblocked kernel is latency- and memory-bound and never approaches the FMA peak. Second, per-matrix LAPACK does the opposite — low at small n, then climbing steeply as blocking and cache reuse engage, overtaking the compact paths and running several times closer to the ceiling at the largest orders; the throughput crossover is,

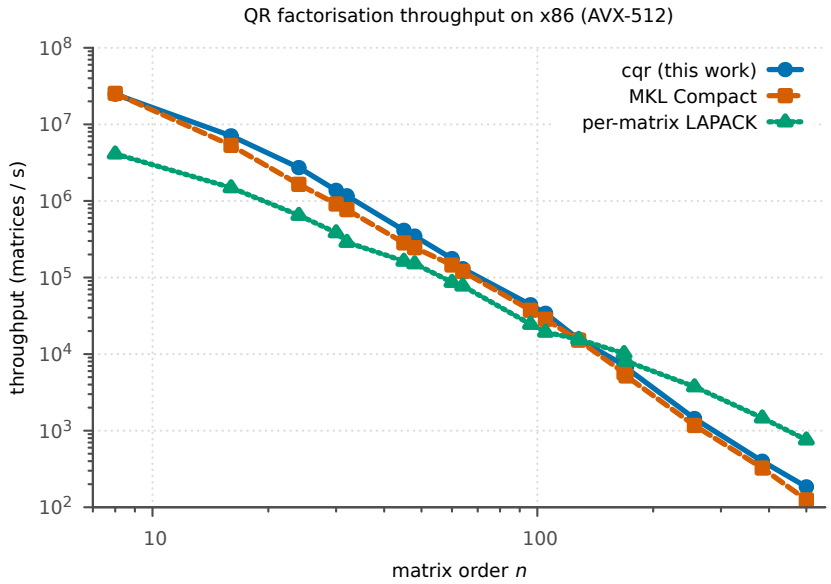

Figure 2: Batched QR factorization throughput on x86 (AVX-512): `cqr` vs MKL Compact vs a per-matrix LAPACK loop, over the target size range. *Measured on a shared VM (provisional).*

in absolute units, this gap opening. Third, the gentle staircase on the `cqr` curve, dipping at orders that are not multiples of the interleave width — the remainder effect of Section 4, and the reason the size list includes the odd stencil sizes.

7 Discussion and limitations

The results (once measured) speak to a specific claim: that the interleaved format's advantage comes from the *layout*, not from any vendor's closed microkernel, and can therefore be had portably. If `cqr` tracks MKL on x86 and ArmPL on Arm from one source, the format — not the implementation — is doing the work, and there is no reason for it to remain closed or ISA-locked.

The scope limits are deliberate and mirror the vendors'. `cqr` is real-only; complex Hermitian QR is future work. Its QR is unpivoted — unlike ArmPL's rank-revealing `?geqrfrr` — so it does not reveal rank, and `cqr` provides neither LU nor GEMM (both deferred to the vendor libraries, Section 3); the target least-squares and interpolation problems are, however, set up to be well conditioned. It omits `larfg`'s underflow/overflow rescaling, matching MKL's documented compact limitation [9]; inputs scaled to the exponent extremes are out of scope. And the row-major and right-apply paths are correctness-first, not separately SIMD-tuned: the tuned path is the column-major, left-apply solver route the applications use.

8 Related work

The batched-BLAS effort defined the problem and a pointer-array interface standard [1, 4]. Two broad strategies then extract the cross-matrix parallelism. The *interleaved/compact* layout — the vendors' answer for the small-matrix regime (Intel Compact [8], Arm interleave-batch [3]) and this paper — rewrites the data so one SIMD instruction advances V matrices. *Kokkos Kernels* [10, 12] reaches the same batched parallelism by different means. Its kernels are C++ templates with the operation properties (transpose, upper/lower, side, and the blocking `Algo`) as compile-time arguments, so each is a header-only `invoke()` that inlines into the user's `parallel_for` and, through Kokkos, runs on CPUs and GPUs alike. Crucially, it still vectorizes *across* matrices when asked: making a `View`'s scalar a SIMD vector type (`Vector<SIMD<T>,L>`)

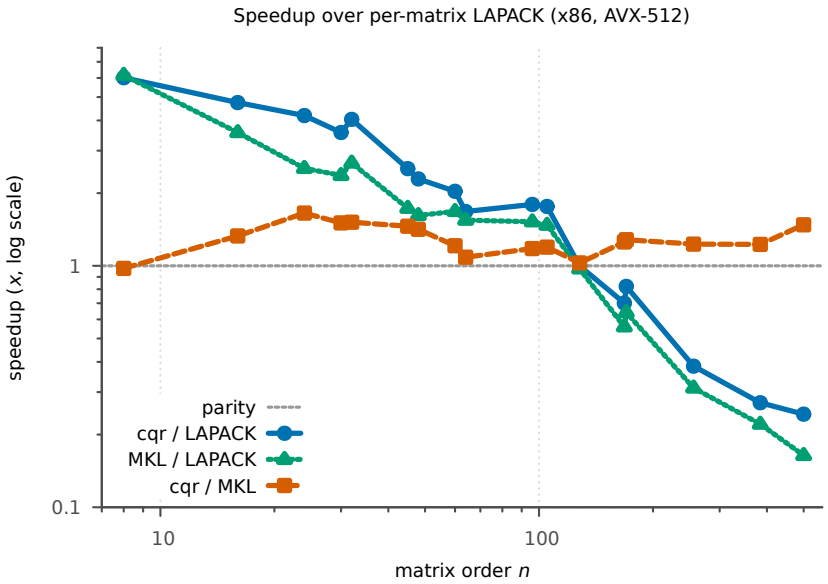

Figure 3: Speedup over the per-matrix LAPACK baseline on x86 (log scale; dashed line is parity). The compact paths win at small orders and cross below parity near $n \approx 128$, where blocked per-matrix LAPACK takes over; `cqr` stays at or above MKL Compact throughout. *Measured on a shared VM (provisional).*

packs L matrices into the lanes, and the same generic kernel advances all L at once — the compact idea expressed through the type system rather than a storage format. That composes with the `Serial`/`Team`/`TeamVector` execution levels (Section 3) and, on GPUs, gives way to them. So the contrast with the compact APIs is not *whether* the batch reaches the SIMD units — it does — but *how* the interleaving is expressed: a SIMD element type reused by generic kernels (Kokkos) versus a dedicated storage format and matching C routines (MKL, ArmPL, `cqr`). `cqr` sits in the latter, format-based camp, as does `batmat` [11] (aimed at convex optimization); its contribution there is narrow and specific: an *open* drop-in for the MKL compact QR, portable across ISAs from one source, that supplies the apply-Q step MKL omits. ▶ [if targeting a venue, add the closest compact-QR performance papers and any batched-QR-on-GPU comparison for context.]

9 Conclusion and future work

The compact/interleaved format is the right data structure for batches of many small identically-shaped matrices, and it is architecture-neutral even though today's implementations are closed and ISA-locked. `cqr` shows the format can be served by a single portable, vector-typed source that stands in for `mkl_?geqrf_compact`, completes the pipeline MKL leaves open, and is meant to match both vendors' kernels in the size range the motivating applications — meshless approximation and multi-physics data mapping — actually use.

Future work: (i) the AArch64/ArmPL benchmark harness needed to turn Figure 4 into measured results — building `cqr` so the vector types lower to NEON/SVE, and a driver that bridges ArmPL's stride-based interleave to the compact packs; (ii) the batched Cholesky `cqr_mkl_?potrf_compact`, already specified, for the normal-equations form of the MLS/RBF problems; (iii) complex precisions; and (iv) an end-to-end application benchmark (an RBF-FD weight assembly) rather than the kernel microbenchmark alone.

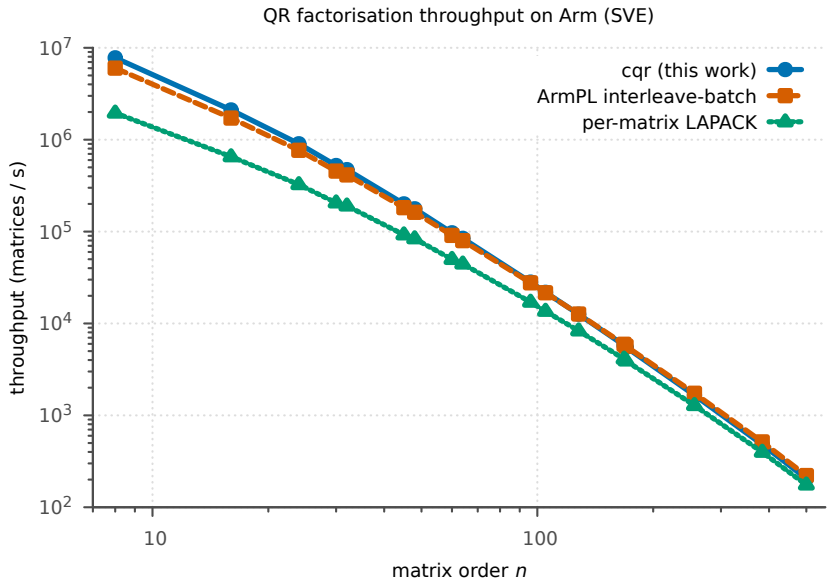

Figure 4: Batched QR factorization throughput on Arm (SVE): `cqr` vs ArmPL interleave-batch vs a per-matrix LAPACK loop. *Placeholder data.*

Availability

▶ [repository URL, licence, and archived release DOI (e.g. Zenodo) for a citable artifact.]

Acknowledgements

▶ [funding, compute resources, and any tool/assistance acknowledgements consistent with the project's conventions.]

References

[1] Ahmad Abdelfattah, Timothy Costa, Jack Dongarra, Mark Gates, Azzam Haidar, Sven Hammarling, Nicholas J. Higham, Jakub Kurzak, Piotr Luszczek, Stanimire Tomov, and Mawussi Zounon. A set of batched basic linear algebra subprograms and LAPACK routines. *ACM Transactions on Mathematical Software*, 47(3), 2021. doi: 10.1145/3431921.

[2] E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, third edition, 1999.

[3] Arm Limited. Arm performance libraries: Interleave-batch functions. https://developer.arm.com/documentation/101004/latest/Interleave-batch-functions, 2026. Accessed 2026.

[4] Batched BLAS Working Group. Batched BLAS (bblas): a proposed standard interface for batched BLAS operations. https://icl.utk.edu/bblas/, 2026. Accessed 2026.

[5] Hans-Joachim Bungartz, Florian Lindner, Bernhard Gatzhammer, Miriam Mehl, Klaudius Scheufele, Alexander Shukaev, and Benjamin Uekermann. preCICE – a fully parallel library for multi-physics surface coupling. *Computers & Fluids*, 141:250–258, 2016. doi: 10.1016/j.compfluid.2016.04.003.

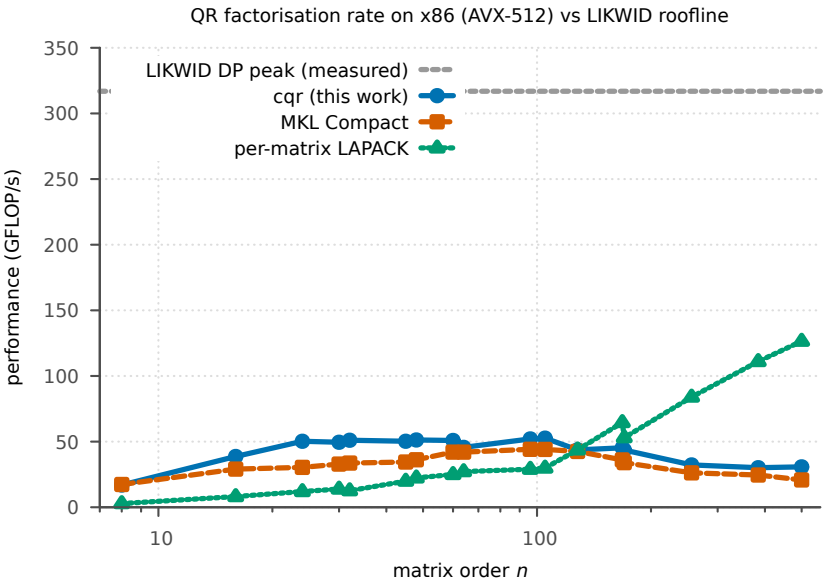

Figure 5: Achieved rate (GFLOP/s) of `cqr`, MKL Compact, and per-matrix LAPACK on x86 (AVX-512), against the LIKWID-measured double-precision peak. *Rates measured on a shared VM (provisional); roofline from LIKWID.*

[6] Bengt Fornberg and Natasha Flyer. *A Primer on Radial Basis Functions with Applications to the Geosciences*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 2015.

[7] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, MD, fourth edition, 2013.

[8] Intel Corporation. oneMKL developer reference (c): Compact BLAS and LAPACK functions. https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/2025-2/compact-blas-and-lapack-functions.html, 2026. Accessed 2026.

[9] Intel Corporation. oneMKL developer reference (c): Numerical limitations of compact BLAS and compact LAPACK functions. https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-c/2025-2/numerical-limits-compact-blas-compact-lapack.html, 2026. Accessed 2026.

[10] Kokkos Kernels developers. Kokkos kernels: Batched API. https://kokkos.org/kokkos-kernels/, 2026. Accessed 2026.

[11] Pieter Pas. batmat: batched small-matrix linear algebra. https://github.com/tttapa/batmat, 2026. Accessed 2026.

[12] Sivasankaran Rajamanickam et al. Kokkos kernels: Performance portable sparse/dense linear algebra and graph kernels. https://arxiv.org/abs/2103.11991, 2021.

[13] Jan Treibig, Georg Hager, and Gerhard Wellein. LIKWID: A lightweight performance-oriented tool suite for x86 multicore environments. In *2010 39th International Conference on Parallel Processing Workshops*, pages 207–216, 2010. doi: 10.1109/ICPPW.2010.38.

[14] Holger Wendland. *Scattered Data Approximation*. Cambridge University Press, 2004.

[15] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

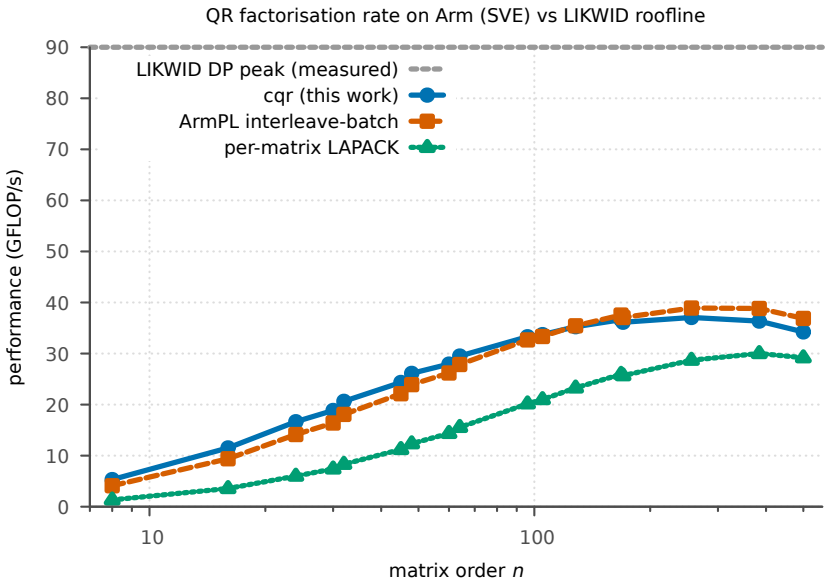

Figure 6: Achieved rate (GFLOP/s) of `cqr`, ArmPL interleave-batch, and per-matrix LAPACK on Arm (SVE), against the LIKWID-measured double-precision peak. *Placeholder data; placeholder roofline.*